

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Proyecto 2 - Reporte Normalización

PRESENTA

Jiménez Rivera Emiliano Kaleb (322049918)

PROFESOR

Amparo López Gaona

ASIGNATURA

Fundamentos de Bases de Datos

15 de diciembre de 2025

Este documento describe el proceso de normalización aplicado al sistema de gestión de aerolíneas, así como restricciones adicionales implementadas para garantizar la integridad y consistencia de los datos, mucho con la teoría vista en clase a lo largo de este semestre. Este proceso se realizó posterior al diseño inicial del modelo entidad-relación y su traducción al modelo relacional, con el objetivo de mejorar la estructura de la base de datos y establecer reglas de negocio menos vulnerables.

1. Renombramiento y Reestructuración de Tablas

La primera fase del proceso de normalización incluyó el renombramiento de tablas y restricciones para mejorar la claridad y descriptividad del esquema. La tabla **Contratar** fue renombrada a **Contratar_aerolinea** para distinguirla explícitamente de la tabla **Contratar_aeropuerto**, eliminando así cualquier ambigüedad en el modelo. Este cambio también implicó la actualización de las restricciones de llave primaria y llaves foráneas asociadas, renombrándolas de **PK_Contratar** a **PK_Contratar_aerolinea**, y de **FK_Contratar_Aerolinea** y **FK_Contratar_Empleado** a **FK_Contratar_Aerolinea_Aero** y **FK_Contratar_Aerolinea_Emp** respectivamente.

Adicionalmente, se modificó el tipo de dato de las columnas **numero_exterior** y **numero_interior** en la tabla **Direccion**, ampliándolos de su tamaño original a **VARCHAR(25)** para soportar formatos de numeración más diversos y facilitar el debugging del sistema, pues en algunas países o regiones usan números extensos e incluso con algunos caracteres.

2. Configuración de Llaves Foráneas

Una parte fundamental de la normalización consistió en la configuración apropiada de las llaves foráneas con diferentes políticas de integridad referencial, dependiendo de las necesidades y requisitos de cada relación.

2.1. Llaves Foráneas con RESTRICT

Se implementó la política **ON DELETE RESTRICT** en la relación entre las tablas **avion** y **aerolineas**. Esta configuración impide la eliminación de una aerolínea que tenga aviones asociados, protegiendo así contra eliminaciones accidentales de aerolíneas en operación. Esta restricción es crítica para mantener la integridad del sistema, ya que la eliminación de una aerolínea con aviones activos podría resultar en inconsistencias graves, incluso legalmente.

De manera similar, la relación entre **piloto_vuelo** y **piloto** utiliza **RESTRICT** para prevenir la eliminación de pilotos que tengan vuelos programados, asegurando que no se pierda información de asignaciones de tripulación.

2.2. Llaves Foráneas con CASCADE

La política **ON DELETE CASCADE** se aplicó en dos relaciones fundamentales del sistema. Primero, entre las tablas **boleto** y **vuelo**, de modo que cuando un vuelo es eliminado del sistema, todos sus boletos asociados se eliminan automáticamente. Esta decisión de diseño

refleja la lógica de negocio donde los boletos no pueden existir independientemente del vuelo al que pertenecen.

Segundo, la relación entre `comprar` y `cliente` también utiliza CASCADE, i.e. cuando un cliente solicita la eliminación de sus datos, todas sus compras se eliminan automáticamente del sistema.

2.3. Llaves Foráneas con SET NULL

La relación entre `vuelo` y `avion` implementa la política `ON DELETE SET NULL`. Cuando un avión es dado de baja, los vuelos que tenía asignados no se eliminan, sino que su campo `matricula_avion` se establece en `NULL`. Esta estrategia permite reasignar vuelos a otros aviones sin perder el registro histórico del vuelo, manteniendo así la trazabilidad operacional, y en sí es una optimización para no repetir mismas asignaciones u operaciones.

3. Restricciones de Dominio

Se establecieron múltiples restricciones de dominio para garantizar la validez y coherencia de los datos almacenados en el sistema.

3.1. Restricciones NOT NULL

Se definieron restricciones de no nulidad en campos críticos del sistema. En la tabla `vuelo`, los campos `fecha_salida` y `hora_salida` fueron marcados como obligatorios, ya que un vuelo debe tener necesariamente definido cuándo despegue. De igual forma, en la tabla `correo_cliente`, el campo `direccion_correo` no puede ser nulo, asegurando que cada cliente tenga al menos un medio de contacto electrónico registrado.

Para la tabla `Contratar_aeropuerto`, se establecieron como obligatorios los campos `Aeropuerto_id`, `Empleado_id` y `Fecha_ingreso`, ya que un registro de contratación sin estos datos carecería de sentido en el contexto del negocio.

4. Normalización a Tercera Forma Normal (3NF)

4.1. Conceptos de Normalización

Primera Forma Normal (1NF)

- Todos los atributos contienen valores atómicos
- No hay grupos repetidos
- Notamos que en el esquema original sí se cumplía

Segunda Forma Normal (2NF)

- Cumple 1NF

- No hay dependencias parciales (en tablas con clave compuesta)
- Todos los atributos no-clave dependen de la clave primaria completa
- Notamos que en el esquema original sí se cumplía

Tercera Forma Normal (3NF)

- Cumple 2NF
- No hay dependencias transitivas
- Todos los atributos no-clave dependen SOLO de la clave primaria
- Pudimos notar que no se cumplía esto por lo que esta sección se dedica a detallar la solución a este problema

4.2. Análisis Inicial y Violaciones Detectadas

Se identificaron **dependencias transitivas** en 4 tablas principales:

1. **Aerolineas** - Dirección con dependencias transitivas
2. **Empleado** - Dirección con dependencias transitivas
3. **Aeropuerto** - Dirección con dependencias transitivas
4. **Vuelo** - Ciudades/países redundantes derivables de aeropuertos, pues al tener aeropuerto_id de llegada y de salida, a partir de ese dato podíamos obtener la dirección de la tabla de aeropuertos.

4.3. Violaciones Detectadas en Detalle

Tabla AEROLINEAS

Dependencia Transitiva Encontrada:

Aerolinea_id → Codigo_postal → Ciudad, Municipio, Pais

Aerolinea_id → Ciudad → Pais

Tabla ANTES (Violación 3NF):

```
CREATE TABLE Aerolineas (  
  Aerolinea_id INT,  
  Razon_social VARCHAR(100) UNIQUE,  
  Pais_origen_empresa VARCHAR(50),  
  -- INICIO PROBLEMA TRANSITIVO  
  Ciudad VARCHAR(50),  
  Municipio VARCHAR(50),  
  Codigo_postal VARCHAR(10),
```

```
Calle VARCHAR(100),  
Colonia VARCHAR(50),  
Numero_exterior VARCHAR(10),  
Pais VARCHAR(50)  
-- FIN PROBLEMA TRANSITIVO  
);
```

Problema: Los campos de dirección dependen del código postal, no directamente de Aerolinea_id.

Impacto en notado en espacio y mantenimiento:

- **Espacio desperdiciado:** Si múltiples aerolíneas comparten la misma dirección (ej: edificio corporativo o alguna estructura particular), los datos de dirección completa se duplican para cada aerolínea. Ejemplo: 10 aerolíneas en el mismo edificio = 10 copias de la misma dirección.
- **Dificultad de mantenimiento:** Si cambia el nombre de una ciudad o se actualiza un código postal, hay que modificar TODAS las filas con esa dirección. Si se olvida actualizar alguna fila, se crean inconsistencias, es decir que no tenemos una fuente de datos única de la que podamos modificar o alterar sin crear inconsistencias.

Tabla DESPUÉS (Cumple 3NF):

```
CREATE TABLE Aerolineas (  
    Aerolinea_id INT,  
    Razon_social VARCHAR(100) UNIQUE,  
    Pais_origen_empresa VARCHAR(50),  
    Direccion_id INT -- Referencia a tabla Direccion  
);
```

Tabla EMPLEADO

Dependencia Transitiva Encontrada:

Empleado_id → Codigo_postal → Ciudad, Municipio, Pais

Problema: Los datos de ubicación geográfica (Pais, Ciudad, Municipio) dependen del Codigo_postal, creando una dependencia transitiva.

Impacto encontrado en espacio y mantenimiento:

- **Espacio desperdiciado:** En una empresa con 500 empleados donde 200 viven en la misma colonia, se almacenan 200 copias idénticas de los campos: Pais, Ciudad, Municipio, Codigo_postal, Colonia. Esto representa aproximadamente 44,000 caracteres redundantes. Es mucho espacio, qué locura.
- **Dificultad de mantenimiento:** Si el gobierno cambia oficialmente el nombre de un municipio o una colonia, es necesario actualizar cientos o miles de registros de empleados individualmente.

Tabla AEROPUERTO

Dependencia Transitiva Encontrada:

Aeropuerto_id → Codigo_postal → Ciudad, Municipio, Pais

Cómo afecta en el espacio y mantenimiento:

- **Espacio desperdiciado:** Aunque los aeropuertos son menos numerosos, aún existe redundancia. Si dos aeropuertos están en la misma ciudad, se duplican todos los datos geográficos.
- **Dificultad de mantenimiento:** Si una ciudad cambia su nombre oficial, hay que actualizar manualmente todos los aeropuertos de esa ciudad.

Tabla VUELO

Dependencia Transitiva Encontrada:

Numero_vuelo → Aeropuerto_salida_id → Ciudad_salida, Pais_salida

Numero_vuelo → Aeropuerto_llegada_id → Ciudad_llegada, Pais_llegada

Problema: Las ciudades y países de salida/llegada ya existen en la tabla Aeropuerto. Almacenarlos aquí crea redundancia y dependencia transitiva.

- **Espacio desperdiciado:** Para una aerolínea con 10,000 vuelos al año entre los mismos aeropuertos recurrentes, se almacenan 10,000 copias de los nombres de ciudades y países. Esto representa aproximadamente 2,000,000 caracteres redundantes.
- **Dificultad de mantenimiento:** Si un aeropuerto cambia de ciudad administrativa o un país cambia de nombre oficial, hay que actualizar TODOS los vuelos históricos y futuros.

4.4. Solución del problema

Nueva Tabla: Direccion

Esta tabla centraliza todos los datos de direcciones, eliminando las dependencias transitivas en todas las tablas mencionadas anteriormente.

```
CREATE TABLE Direccion (  
  Direccion_id SERIAL PRIMARY KEY,  
  Pais VARCHAR(50) NOT NULL,  
  Ciudad VARCHAR(50),  
  Municipio VARCHAR(50),  
  Codigo_postal VARCHAR(20),  
  Calle VARCHAR(100),  
  Colonia VARCHAR(50),  
  Numero_exterior VARCHAR(10),  
  Numero_interior VARCHAR(10)  
);
```

Dependencias Funcionales (3NF):

Direccion_id → Pais, Ciudad, Municipio, Codigo_postal,
Calle, Colonia, Numero_exterior, Numero_interior

Así notamos que elimina duplicación de direcciones, dependencias transitivas, también facilita actualización de direcciones (un solo lugar) y permite reutilización de direcciones.

4.5. Impacto en Consultas

Esta sección es importante pues es donde podemos ejemplificar mejor por qué la normalización es un concepto importante en la teoría de bases de datos y lo que nos facilita de trabajo al aplicarla.

ANTES de la normalización

Consulta simple - Información de vuelo:

```
SELECT
    v.Numero_vuelo,
    v.Ciudad_salida,
    v.Pais_salida,
    v.Ciudad_llegada,
    v.Pais_llegada,
    v.Fecha_salida
FROM Vuelo v
WHERE v.Numero_vuelo = 'AA123';
```

Rendimiento: Rápido (sin JOINS)

Problema: Datos duplicados, violación 3NF

DESPUÉS de la normalización

Consulta equivalente - Información de vuelo:

```
SELECT
    v.Numero_vuelo,
    d_salida.Ciudad AS Ciudad_salida,
    d_salida.Pais AS Pais_salida,
    d_llegada.Ciudad AS Ciudad_llegada,
    d_llegada.Pais AS Pais_llegada,
    v.Fecha_salida
FROM Vuelo v
INNER JOIN Aeropuerto a_salida
    ON v.Aeropuerto_salida_id = a_salida.Aeropuerto_id
INNER JOIN Direccion d_salida
    ON a_salida.Direccion_id = d_salida.Direccion_id
INNER JOIN Aeropuerto a_llegada
    ON v.Aeropuerto_llegada_id = a_llegada.Aeropuerto_id
INNER JOIN Direccion d_llegada
```

```
ON a_llegada.Direccion_id = d_llegada.Direccion_id
WHERE v.Numero_vuelo = 'AA123';
```

Rendimiento: Requiere 4 JOINS

Beneficio: Cumple 3NF, sin redundancia

4.6. Beneficios de la Normalización

Por último para cerrar esta sección mencionaremos algunas de las características que notamos a lo largo de la realización del proyecto. **Ventajas**

- **Integridad de Datos:** i.e. eliminación de redundancia, tenemos una sola fuente de verdad para direcciones y las actualizaciones son consistentes.
- **Mantenibilidad:** Cambios de dirección en un solo lugar y hay menos riesgo de inconsistencias.
- **Espacio en Disco:** reducción de datos duplicados y direcciones compartibles entre entidades
- **Verificamos y demostramos que nuestra base de datos está en forma de 3NF**
 - Base de datos correctamente normalizada
 - Sin dependencias transitivas

5. Validación de Restricciones Complejas

Se implementaron restricciones adicionales para validar reglas de negocio más complejas. Se estableció validación para el origen de créditos, asegurando que solo provengan de fuentes válidas como cancelación de vuelo, compensación, promoción o devolución. El tipo de notificación debe pertenecer a categorías predefinidas: retraso, cancelación, cambio de puerta, recordatorio o alerta. La clase de boleto se limita a Primera, Ejecutiva o Económica, reflejando las categorías estándar de la industria aérea, entre muchas otras que están documentadas en el código fuente del proyecto.

5.1. Notas extras: restricciones CHECK

Se implementaron numerosas restricciones CHECK para validar rangos y valores lógicos. La capacidad de pasajeros de un avión debe estar entre cero y mil asientos, considerando que el Airbus A380, el avión comercial más grande en operación, tiene aproximadamente ochocientos cincuenta plazas. El valor cero se permite para aviones de carga que no transportan pasajeros, i.e. aquellos de carga.

Los precios de boletos deben ser estrictamente positivos y no exceder un millón de unidades monetarias, eliminando la posibilidad de boletos gratuitos o con precios negativos que no tendrían sentido en el modelo de negocio.

Los salarios de empleados se restringen al rango de cinco mil a cinco millones de unidades monetarias, contemplando diferentes posiciones y monedas internacionales. Esta validación previene errores de captura que podrían resultar en salarios ilógicos.

Las horas de vuelo de un piloto deben ser no negativas y no exceder cincuenta mil horas, un límite razonable considerando una carrera completa en la aviación comercial.

Se implementó una restricción compleja para validar que la fecha y hora de llegada de un vuelo sean posteriores a la fecha y hora de salida, evitando inconsistencias temporales imposibles en el mundo real.

La duración de vuelos en minutos debe ser positiva y no exceder mil quinientos minutos (veinticinco horas), cubriendo incluso los vuelos comerciales más largos del mundo.

El estado del vuelo debe pertenecer al conjunto de valores válidos: programado, abordando, en vuelo, aterrizado, cancelado o retrasado. De manera similar, el estado del boleto se restringe a: activo, cancelado, reembolsado o usado.

Las fechas de nacimiento de empleados deben asegurar que tengan al menos dieciocho años de edad legal para trabajar, y no pueden ser futuras ni anteriormente a 1940, estableciendo un rango razonable para fechas de nacimiento.

La fecha de último mantenimiento de un avión no puede ser futura, asegurando que solo se registren mantenimientos ya realizados.

La fecha de egreso de un empleado de un aeropuerto debe ser posterior a su fecha de ingreso, manteniendo la coherencia temporal en los registros laborales.

6. Lo que nos llevamos de este proyecto

El proceso de normalización aplicado al sistema de gestión de aerolíneas ha resultado en un esquema de base de datos más robusto, coherente y alineado con las buenas prácticas de diseño de bases de datos relacionales. Las restricciones implementadas no solo garantizan la integridad de los datos, sino que también refuerzan las reglas de negocio (o requisitos que algún cliente tenga) y facilitan el mantenimiento futuro del sistema. La cuidadosa configuración de llaves foráneas con diferentes políticas de eliminación nos proporciona la versatilidad o flexibilidad necesaria para manejar diversos escenarios, mientras mantiene la consistencia de los datos como se vio en el proceso de la tabla Dirección.